Guida al clustering gerarchico

Dal codice al dendrogramma

Questa guida accompagna l’esercitazione della cartella **04_clustering_gerarchico**. Si parte dai nove punti della traccia, si segue l’algoritmo manuale e si riproduce il risultato con SciPy. Il confronto finale distingue single linkage e metodo del baricentro.

Prerequisiti Liste, tuple, insiemi, dizionari, funzioni, if, for e while. La soluzione manuale usa soltanto Python 3; l’alternativa richiede SciPy e Matplotlib.

Uso in aula Prevedere la prossima fusione, leggere il blocco di codice ed eseguirlo. Il codice eseguibile dei due script è riportato per blocchi consecutivi; le spiegazioni sostituiscono i commenti estesi presenti nei file del progetto.

Indice

Argomento	Pagina
Dati e corrispondenza fra indici ed etichette	2
Distanza euclidea fra punti	3
Distanza fra cluster e linkage	4
Foglie genitori ed etichette	5
Inizializzazione dei cluster attivi	6
Ricerca dei due cluster più vicini	7
Fusione aggiornamento e stampa	8
Esecuzione manuale e confronto con la traccia	9
Metodo del baricentro e confronto dei criteri	10
SciPy import e opzioni di esecuzione	11
SciPy matrice Z e stampa delle fusioni	12
SciPy disegno e salvataggio del grafico	13
Dendrogrammi single e centroid a confronto	14
Domande esercizi e riferimenti	15

Dati e corrispondenza fra indici ed etichette

Ogni punto è una tupla (x, y). POINTS è una lista ordinata di nove tuple; LABELS contiene i nomi nello stesso ordine. I cluster conserveranno gli indici dei punti: le etichette servono per visualizzare il risultato.

```
import math

POINTS = [
    (12, 23),
    (25, 20),
    (16, 27),
    (81, 26),
    (55, 21),
    (70, 35),
    (18, 52),
    (48, 64),
    (51, 69)
]

LABELS = ["A", "B", "C", "D", "E", "F", "G", "H", "I"]
```

Tre modi per identificare un punto

Indice Python	Etichetta	Numero nella traccia
0	A	1
1	B	2
2	C	3
3	D	4
4	E	5
5	F	6
6	G	7
7	H	8
8	I	9

POINTS[0] restituisce (12, 23); LABELS[0] restituisce "A". L’insieme di membri {0, 2} rappresenta A e C, cioè i punti 1 e 3 della traccia. I nomi dei membri vengono ordinati secondo gli indici prima della stampa.

Domanda in aula Quali etichette corrispondono a {3, 5}?
Risposta D e F, cioè i punti 4 e 6. L’import math è conservato dal sorgente originale, ma la radice viene calcolata con l’operatore **.

Distanza euclidea fra punti

La distanza misura quanto due punti sono lontani. In due dimensioni si calcolano le differenze delle coordinate, si elevano al quadrato, si sommano e si prende la radice. La funzione estende lo stesso procedimento a qualsiasi numero di coordinate, purché uguale per entrambi i punti.

```
def distanza_euclidea(p, q):

    somma_quadrati = 0.0

    for i in range(len(p)):

        differenza = p[i] - q[i]

        somma_quadrati += differenza * differenza

    return somma_quadrati ** 0.5
```

Seguire l’accumulatore

somma_quadrati parte da 0.0. range(len(p)) genera gli indici delle coordinate. A ogni iterazione, += aggiunge il quadrato della differenza; ** 0.5 restituisce la radice quadrata della somma. return conclude la funzione e consegna il risultato al chiamante.

Esempio con A e C

```
A = (12, 23)
C = (16, 27)
# Differenze: 12 - 16 = -4 e 23 - 27 = -4
# Quadrati: 16 + 16 = 32
# Distanza: sqrt(32) = circa 5.656854
print(distanza_euclidea(A, C))
```

La traccia riporta **32**, cioè il quadrato della distanza. Il programma stampa la distanza euclidea arrotondata a due decimali: **5.66**. Il calcolo usa il valore non arrotondato.

Distinguere calcolo e visualizzazione

Per il single linkage, prendere la radice quadrata non cambia l’ordine dei minimi. Per l’average linkage, invece, la media delle distanze non è la radice della media dei quadrati: non sostituire automaticamente una rappresentazione con l’altra.

Domanda in aula Perché la differenza negativa non produce una distanza negativa?
Risposta Il quadrato di ciascuna differenza è non negativo; anche la radice della somma è non negativa.

Distanza fra cluster e linkage

Un cluster può contenere più punti. Il criterio di linkage stabilisce come ottenere una sola distanza da tutte le coppie incrociate, una con un punto nel primo cluster e un punto nel secondo.

```
def cluster_distance(c1, c2, points, linkage):

    if linkage == "single":

        distanza_minima = float("inf")

        for i in c1["members"]:
            punto1 = points[i]
            for j in c2["members"]:
                punto2 = points[j]
                distanza = distanza_euclidea(punto1, punto2)

                if distanza < distanza_minima:
                    distanza_minima = distanza

        return distanza_minima

    elif linkage == "average":

        somma_distanze = 0.0
        numero_coppie = 0

        for i in c1["members"]:
            punto1 = points[i]
            for j in c2["members"]:
                punto2 = points[j]
                distanza = distanza_euclidea(punto1, punto2)

                numero_coppie += 1
                somma_distanze += distanza

        return somma_distanze / numero_coppie

    return None
```

Single float("inf") inizializza il minimo a infinito; una distanza finita può sostituirlo. I cicli annidati esaminano tutte le coppie e l'if conserva la più piccola.

Average Si sommano le distanze e si contano le coppie. Con m punti nel primo cluster e n nel secondo, il denominatore è m × n. I cluster prodotti dall'algoritmo sono non vuoti.

Domanda in aula Quante coppie si esaminano fra cluster di 2 e 3 punti?
Risposta Sei. Passare un metodo diverso da single o average restituisce None: il chiamante non può confrontarlo con un numero, quindi il metodo deve essere riconosciuto.

Foglie genitori ed etichette

Ogni nodo è un dizionario. Una foglia rappresenta un punto; un genitore conserva i due figli e l'unione dei membri. Il formato delle stampe viene gestito separatamente dai calcoli.

```
def make_leaf(idx):

    return {
        "id": idx,
        "members": {idx},
        "height": 0.0,
        "size": 1
    }

def make_parent(new_idx, left, right, height):

    return {
        "id": new_idx,
        "left": left,
        "right": right,
        "members": left["members"] | right["members"],
        "height": height,
        "size": left["size"] + right["size"]
    }

def format_cluster(cluster, labels=None):

    indici = sorted(cluster["members"])
    if labels is None:
        nomi = [str(i) for i in indici]
    else:
        nomi = [str(labels[i]) for i in indici]

    return "{" + ", ".join(nomi) + "}"
```

Campo	Significato
id	Identificatore stabile del nodo, diverso dalla posizione nella lista.
members	Insieme degli indici dei punti contenuti nel nodo.
height	Distanza della fusione; zero per una foglia.
size	Numero di punti, somma delle dimensioni dei figli.
left e right	Riferimenti ai nodi figli, presenti soltanto nei genitori.

Da evidenziare {idx} è un insieme; | unisce due insiemi. sorted ordina gli indici prima di trasformarli in nomi e join li separa con virgole. Senza labels la funzione mostra gli indici numerici.

Inizializzazione dei cluster attivi

La funzione principale riceve dati, metodo, opzione di stampa ed eventuali etichette. Questo blocco apre la funzione; le pagine 7 e 8 continuano lo stesso corpo mantenendo l’indentazione.

```
def hierarchical_clustering(points, linkage_method="single", verbose=True, labels=None):

    if labels is not None and len(labels) != len(points):
        raise ValueError("Occorre un'etichetta per ogni punto")

    clusters = [make_leaf(i) for i in range(len(points))]
    step = 0

    if verbose:
        stato = " ".join(format_cluster(c, labels) for c in clusters)
        print(f"Step {step}: {stato}")
```

Parametri e valori predefiniti

points contiene i punti. **linkage_method** vale single se non specificato. **verbose** abilita o disabilita le stampe. **labels** è facoltativo: se fornito deve contenere un’etichetta per ciascun punto, altrimenti viene sollevato ValueError.

Una foglia per ogni punto

La list comprehension [make_leaf(i) for i in range(len(points))] costruisce nove foglie. La lista clusters contiene soltanto i nodi ancora attivi: a ogni fusione due di questi nodi vengono sostituiti da un genitore.

step parte da zero. format_cluster rende leggibili i membri; il generatore dentro join produce una rappresentazione per ciascun cluster attivo. Le etichette cambiano la stampa, mentre members mantiene gli indici originali.

```
Step 0: {A} {B} {C} {D} {E} {F} {G} {H} {I}
```

Domanda in aula Quanti nodi e quante fusioni ci sono alla partenza?
Risposta Nove foglie attive e nessuna fusione. Per arrivare a una sola radice occorrono otto fusioni.

Ricerca dei due cluster più vicini

Continuazione di hierarchical_clustering. Il while esegue una fusione per volta, finché rimane più di un cluster. A ogni iterazione la ricerca del minimo riparte da capo.

```
    while len(clusters) > 1:

        best_i = None
        best_j = None
        best_distance = float("inf")

        for i in range(len(clusters)):
            for j in range(i + 1, len(clusters)):
                distanza = cluster_distance(
                    clusters[i], clusters[j], points, linkage_method
                )

                if distanza < best_distance:
                    best_distance = distanza
                    best_i = i
                    best_j = j
```

Perché j parte da i più uno

I due for esaminano tutte le coppie di cluster distinti. j parte da i + 1, quindi non si confronta un cluster con se stesso e non si ripete la stessa coppia nell’ordine inverso. Con k cluster si fanno k × (k - 1) / 2 confronti.

Posizioni e identificatori

best_i e best_j sono posizioni nella lista clusters, utili per recuperare e rimuovere i nodi. Non sono gli id dei nodi e non identificano direttamente i punti. best_distance conserva la distanza minima trovata.

Il confronto usa <: se due coppie hanno la stessa distanza rimane la prima incontrata nell’ordine corrente della lista. La distanza tra cluster viene ricalcolata dai loro membri; questa soluzione non conserva una matrice delle distanze.

Domanda in aula Quanti confronti tra cluster si fanno con quattro cluster attivi?
Risposta Sei. Ogni confronto può a sua volta richiedere molte distanze fra punti, calcolate dentro cluster_distance.

Fusione aggiornamento e stampa

Continuazione e conclusione di hierarchical_clustering. La coppia selezionata viene collegata al nuovo genitore; i due figli escono dalla lista attiva, ma restano raggiungibili attraverso il genitore.

```
        cluster1 = clusters[best_i]
        cluster2 = clusters[best_j]

        new_id = len(points) + step
        parent_cluster = make_parent(new_id, cluster1, cluster2, best_distance)

        for idx in sorted([best_i, best_j], reverse=True):
            clusters.pop(idx)
        clusters.append(parent_cluster)

        step += 1
        if verbose:
            stato = " ".join(format_cluster(c, labels) for c in clusters)
            print(f"Step {step}: {stato}")

            print(f"  -> Uniti cluster {format_cluster(cluster1, labels)} "
                  f"e {format_cluster(cluster2, labels)} "
                  f"(distanza: {best_distance:.2f})")

    return clusters[0]
```

Rimuovere prima l’indice maggiore

pop elimina una posizione dalla lista e sposta gli elementi successivi. Se si vogliono rimuovere le posizioni 1 e 4, togliendo prima 1 la vecchia posizione 4 diventa 3. sorted(..., reverse=True) fa rimuovere prima 4 e poi 1.

Una fusione riduce il numero di cluster

Si rimuovono due cluster e se ne aggiunge uno: il numero di cluster attivi diminuisce di uno. I genitori ricevono id da n in poi. Il contatore step aumenta dopo la fusione; :.2f formatta le distanze con due decimali senza arrotondare il valore usato nei calcoli.

return clusters[0] restituisce la radice completa. La funzione assume un dataset non vuoto e un metodo supportato; con un solo punto il while non parte e si restituisce la foglia.

Domanda in aula I figli sono perduti quando si esegue pop?
Risposta No: left e right del genitore conservano i riferimenti ai loro dizionari.

Esecuzione manuale e confronto con la traccia

Il blocco finale si esegue solo avviando direttamente dendrogram.py. Importandolo nello script SciPy, dati e funzioni diventano disponibili senza lanciare automaticamente il clustering.

```
if __name__ == "__main__":
    print("=" * 70)
    print("CLUSTERING GERARCHICO AGGLOMERATIVO")
    print("=" * 70)
    print(f"\nNumero di punti: {len(POINTS)}")
    print(f"Metodo di linkage: Single Linkage\n")

    root = hierarchical_clustering(
        POINTS, linkage_method="single", verbose=True, labels=LABELS
    )

    print("\n" + "=" * 70)
    print("CLUSTERING COMPLETATO")
    print("=" * 70)

    print(f"\nCluster finale: {format_cluster(root, LABELS)}")
    print(f"Altezza totale del dendrogramma: {root['height']:.2f}")
```

Avvio dalla cartella principale python3 04_clustering_gerarchico/dendrogram.py

Fusione	Cluster creato	Distanza circa	Quadrato nella traccia
1	AC	5.66	32
2	HI	5.83	34
3	ABC	11.40	130
4	DF	14.21	202
5	DEF	20.52	421
6	ABCG	25.08	629
7	ABCDEFG	30.02	901
8	ABCDEFGHI	32.31	1044

Il cluster finale è {A, B, C, D, E, F, G, H, I}; l’altezza è sqrt(1044), stampata come 32.31. I quadrati della tabella derivano dalle distanze non arrotondate. Le otto fusioni coincidono con l’esempio svolto nella traccia.

Metodo del baricentro e confronto dei criteri

La traccia chiede di ripetere l’esercizio usando i baricentri. Un baricentro si ottiene facendo la media delle coordinate di tutti i punti del cluster. La distanza tra baricentri è un criterio diverso dalla media delle distanze fra coppie.

Metodo	Come definisce la distanza tra due cluster
single	Minimo delle distanze delle coppie incrociate.
complete	Massimo delle distanze delle coppie incrociate.
average	Media delle distanze delle coppie incrociate.
centroid	Distanza euclidea fra i baricentri dei cluster.

Esempio con il cluster AC e il punto B

A = (12, 23) e C = (16, 27), quindi il loro baricentro è ((12 + 16) / 2, (23 + 27) / 2) = **(14, 25)**. B = (25, 20). Le distanze B-A e B-C sono sqrt(178) e sqrt(130).

Metodo	Calcolo per AC e B	Risultato circa
single	min(sqrt(178), sqrt(130))	11.40
complete	max(sqrt(178), sqrt(130))	13.34
average	(sqrt(178) + sqrt(130)) / 2	12.37
centroid	sqrt((25-14)^2 + (20-25)^2)	12.08

Il peso dei punti conta

Il baricentro di ABC è ((12 + 25 + 16) / 3, (23 + 20 + 27) / 3), circa (17.67, 23.33). Per ottenerlo dai baricentri dei figli, AC deve avere peso 2 e B peso 1: non basta mediare i due baricentri con lo stesso peso.

Come eseguire l’esercizio

La soluzione manuale supporta single e average. Lo script alternativo supporta anche complete e centroid. Per il baricentro usare **--metodo centroid**, passando a linkage le coordinate originali con metrica euclidea.

Da discutere Sul dataset fornito, single e centroid condividono le prime fusioni ma hanno altezze diverse; dopo sei fusioni cambia anche la struttura dei gruppi. Il confronto grafico è a pagina 14.

SciPy import e opzioni di esecuzione

Il codice di dendrogram_scipy.py comincia importando gli stessi dati della soluzione manuale. Le funzioni della libreria calcolano e disegnano il clustering; argparse permette di scegliere il criterio senza modificare il sorgente.

```
import argparse
from pathlib import Path

import matplotlib
from scipy.cluster.hierarchy import dendrogram, linkage

from dendrogram import LABELS, POINTS

def main():

    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--metodo",
        choices=["single", "complete", "average", "centroid"],
        default="single",
        help="Criterio di fusione dei cluster (predefinito: single)",
    )
    parser.add_argument(
        "--no-show",
        action="store_true",
        help="Salva il grafico senza aprire una finestra",
    )
    args = parser.parse_args()

    if args.no_show:
        matplotlib.use("Agg")
    import matplotlib.pyplot as plt
```

Preparare le dipendenze

```
source .venv/bin/activate
python3 -m pip install -r 04_clustering_gerarchico/requirements.txt
python3 04_clustering_gerarchico/dendrogram_scipy.py
```

Comandi per macOS/Linux dalla cartella principale. In PowerShell su Windows l’attivazione è .venv\Scripts\Activate.ps1. Se non si usa un ambiente virtuale, installare le dipendenze nell’interprete scelto per eseguire la lezione.

Opzioni --metodo sceglie single, complete, average o centroid. --no-show imposta il backend Agg prima dell’import di pyplot: il grafico viene salvato senza aprire una finestra.

SciPy matrice Z e stampa delle fusioni

Continuazione di main. linkage sostituisce la ricerca e le fusioni scritte a mano. Restituisce Z, una matrice di n - 1 righe e quattro colonne. Si passano i punti, non la matrice dei quadrati delle distanze della traccia.

```
    Z = linkage(POINTS, method=args.metodo, metric="euclidean")

    print(f"Metodo: {args.metodo}; punti: {len(POINTS)}")
    print("\nMatrice delle fusioni Z:")
    print(Z)

    membri = {i: [LABELS[i]] for i in range(len(POINTS))}
    print("\nFusioni con le etichette dei punti:")
    for passo, riga in enumerate(Z, start=1):
        sinistro, destro, distanza, dimensione = riga
        sinistro = int(sinistro)
        destro = int(destro)
        nuovo_id = len(POINTS) + passo - 1
        membri[nuovo_id] = sorted(membri[sinistro] + membri[destro])
        print(
            f"Passo {passo}: {membri[sinistro]} + {membri[destro]} "
            f"-> {membri[nuovo_id]} ({int(dimensione)} punti); "
            f"d = {distanza:.2f}; d^2 = {distanza ** 2:.2f}"
        )
```

Colonna	Significato
Z[i, 0]	ID del primo cluster figlio.
Z[i, 1]	ID del secondo cluster figlio.
Z[i, 2]	Distanza della fusione.
Z[i, 3]	Numero di punti nel nuovo cluster.

Le foglie hanno id da 0 a 8. La riga i crea il genitore con id 9 + i. Gli id contenuti in Z sono rappresentati come float: int li converte per usarli come chiavi. enumerate inizia il numero di passo da 1, perciò il codice usa len(POINTS) + passo - 1.

Etichette membri parte dalle singole lettere e aggiunge la lista dei membri per ciascun genitore. Nel single linkage, la prima riga è circa [0, 2, 5.656854, 2]: A e C creano il nodo 9, contenente due punti.

SciPy disegno e salvataggio del grafico

Conclusione di main e avvio dello script. dendrogram interpreta la matrice Z e disegna i collegamenti tra figli e genitori. labels assegna alle foglie i nomi A-I.

```
    figura, asse = plt.subplots(figsize=(9, 5))
    dendrogram(Z, labels=LABELS, ax=asse)
    asse.set_title(f"Clustering gerarchico - {args.metodo} linkage")
    asse.set_xlabel("Punti")
    asse.set_ylabel("Distanza euclidea")
    figura.tight_layout()

    cartella = Path(__file__).resolve().parent / "grafici_generati"
    cartella.mkdir(parents=True, exist_ok=True)
    percorso = cartella / f"dendrogramma_{args.metodo}.png"
    figura.savefig(percorso, dpi=180)
    print(f"\nGrafico salvato in: {percorso}")

    if not args.no_show:
        plt.show()
    plt.close(figura)

if __name__ == "__main__":
    main()
```

Leggere il risultato

L’asse verticale mostra la distanza della fusione. L’asse orizzontale dispone le foglie per rendere leggibile l’albero: non rappresenta la coordinata x dei punti. Scambiare sinistra e destra di un ramo conserva i gruppi e le altezze, quindi un disegno può apparire diverso dalla traccia pur descrivendo lo stesso clustering.

Il file PNG viene salvato in **04_clustering_gerarchico/grafici_generati**. Path(__file__).resolve().parent rende il percorso indipendente dalla directory di lavoro. mkdir prepara la cartella e savefig salva a 180 dpi. Ripetere lo stesso metodo sovrascrive il relativo PNG.

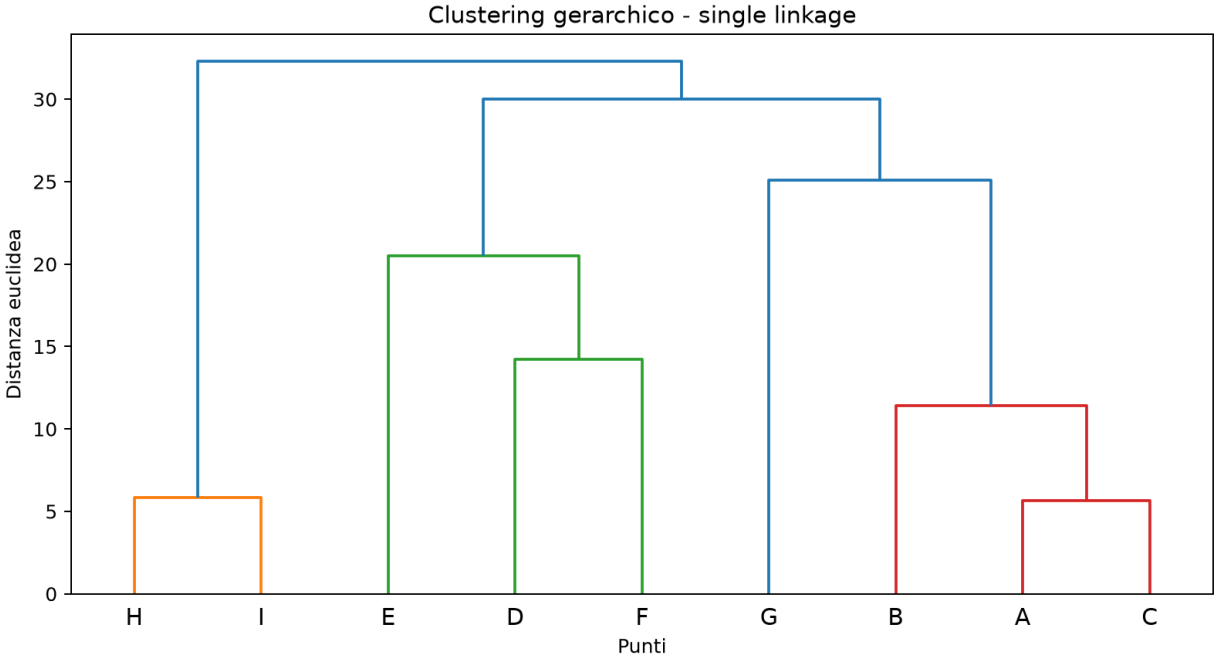
Comandi per il confronto

```
python3 04_clustering_gerarchico/dendrogram_scipy.py --metodo single
python3 04_clustering_gerarchico/dendrogram_scipy.py --metodo centroid
python3 04_clustering_gerarchico/dendrogram_scipy.py --no-show
```

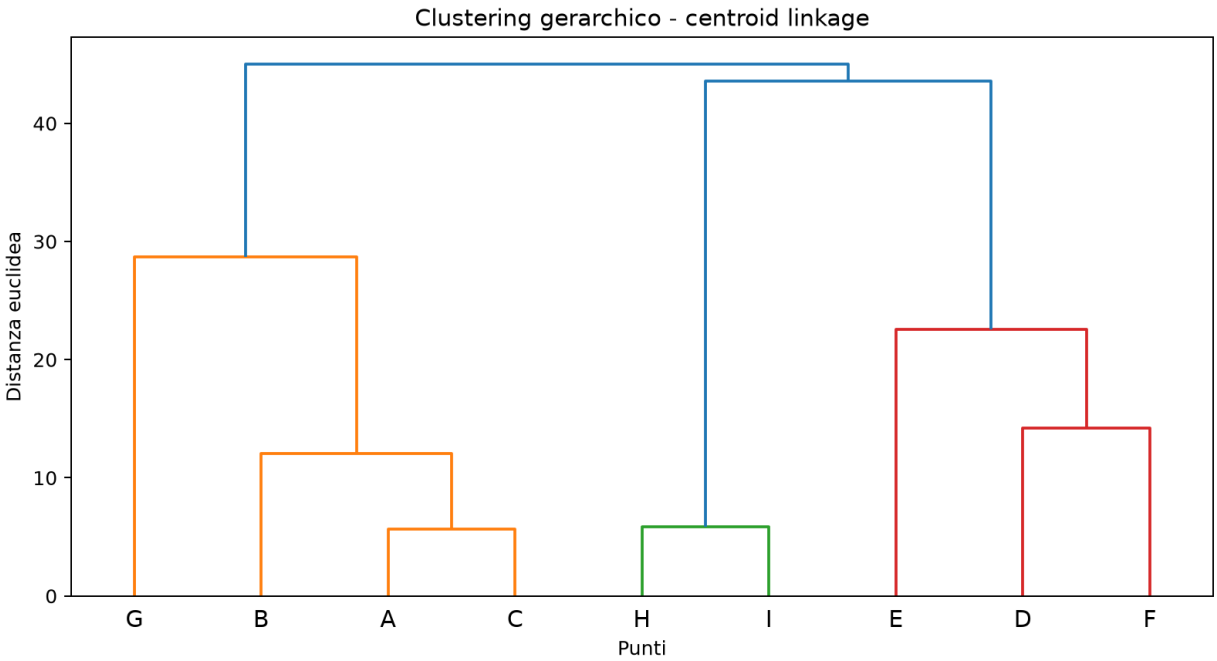
Domanda in aula --no-show cambia il clustering?
Risposta No: cambia soltanto la modalità di visualizzazione. Il file viene comunque salvato; plt.close libera la figura dopo l’uso.

Dendrogrammi single e centroid a confronto

Entrambi i grafici usano distanze euclidee. Le lettere identificano gli stessi nove punti; leggere i rami dal basso verso l'alto. Le diverse posizioni orizzontali non indicano da sole un cambiamento nei cluster.



Single Alla fusione 7, ABCG si unisce con DEF a distanza 30.02; HI entra soltanto alla fusione 8, a distanza 32.31.



Centroid Alla fusione 7, DEF si unisce con HI a distanza 43.60; ABCG entra alla fusione 8, a distanza 45.02.

Confronto Tagliando entrambi gli alberi a distanza 31 si ottengono due cluster nel single linkage e tre nel centroid. Cambiare criterio può cambiare gruppi e distanze delle fusioni. I valori sono calcolati dai nove punti forniti.

Domande esercizi e riferimenti

Chiedere di prevedere il risultato prima dell'esecuzione e di motivare le risposte usando i dati o una riga del codice. Le attività collegano l'algoritmo alle strutture Python già introdotte.

1. Tradurre {0, 2, 6} in etichette e numeri della traccia.
Verifica Etichette A, C, G; numeri 1, 3, 7.

2. Calcolare la prima fusione single linkage e la sua altezza.
Verifica A e C: distanza sqrt(32), circa 5.66.

3. Spiegare perché servono otto fusioni per nove punti.
Verifica Ogni fusione riduce di uno i cluster attivi: 9 - 8 = 1.

4. Simulare la rimozione delle posizioni 1 e 4 da una lista.
Verifica Rimuovere prima 4: pop(1) sposta le posizioni successive.

5. Eseguire il clustering con verbose=False e labels=LABELS.
Verifica La funzione non stampa i passi, ma restituisce lo stesso albero.

6. Confrontare average e centroid sulla coppia AC-B.
Verifica Average circa 12.37; centroid circa 12.08. Spiegare quali quantità vengono mediate.

7. Modificare una sola etichetta senza cambiare le coordinate.
Verifica Cambiano i nomi nell'output, non le distanze o le fusioni.

8. Eseguire single e centroid e tagliare gli alberi a distanza 31.
Verifica Single: ABCDEFG e HI. Centroid: ABCG, DEF e HI.

9. Aggiungere complete alla soluzione manuale come esercizio.
Verifica Usare il massimo fra tutte le distanze delle coppie incrociate e confrontare con SciPy.

File da tenere accanto alla guida

Cartella 04_clustering_gerarchico: **Traccia.pdf**, **dendrogram.py**, **dendrogram_scipy.py** e **requirements.txt**. Il codice della guida segue i file aggiornati con la stampa delle etichette A-I.

Riferimenti SciPy

Documentazione ufficiale di linkage:
docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.linkage.html

Documentazione ufficiale di dendrogram:
docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.dendrogram.html